Lesson 05 Walkthrough: SQL Aggregations & GROUP BY

Database Applications Development

Welcome! This walkthrough demonstrates SQL aggregate functions and grouping with the NBA dataset.

What You'll Learn:

1. Aggregate functions: COUNT, SUM, AVG, MIN, MAX
2. GROUP BY for category analysis
3. HAVING to filter groups
4. Exporting results to Excel

Prerequisites:

- ✅ Completed Lessons 01-04
- ✅ NBA database created
- ✅ Understand basic SQL SELECT queries

Let's analyze some NBA data!

NBA SQLite Data Dictionary

Tables

- `players`
- `teams`
- `player_season_stats`
- `team_game_stats`

Keys and Relationships

- `players.player_id` → referenced by `player_season_stats.player_id`
- `teams.team_id` → referenced by `player_season_stats.team_id` and `team_game_stats.team_id`
- `team_game_stats` uses a **composite primary key**: (`season`, `game_id`, `team_id`)

- `player_season_stats` uses a **composite primary key**: (`season` , `player_id` , `team_id`)

`players`

Description: One row per player.

Column	Type	Key	Description
`player_id`	INTEGER	PK	Unique player identifier
`full_name`	TEXT		Player full name

`teams`

Description: One row per NBA team.

Column	Type	Key	Description
`team_id`	INTEGER	PK	Unique team identifier
`full_name`	TEXT		Team full name (e.g., “Cleveland Cavaliers”)
`abbreviation`	TEXT		Team abbreviation (e.g., “CLE”)
`nickname`	TEXT		Team nickname/mascot name (e.g., “Cavaliers”)
`city`	TEXT		Team city
`state`	TEXT		Team state
`year_founded`	INTEGER		Year the franchise was founded

`player_season_stats`

Description: Per-player season stats (one row per `player_id` + `team_id` + `season`).

Column	Type	Key	Description
`season`	TEXT	PK*	Season label (format depends on your source, e.g., `2019–20`)
`player_id`	INTEGER	PK* / FK	Player ID → `players.player_id`

Column	Type	Key	Description
`team_id`	INTEGER	PK* / FK	Team ID → `teams.team_id`
`gp`	INTEGER		Games played
`min`	REAL		Minutes per game (often per-game average)
`pts`	REAL		Points per game
`reb`	REAL		Rebounds per game
`ast`	REAL		Assists per game
`stl`	REAL		Steals per game
`blk`	REAL		Blocks per game
`tov`	REAL		Turnovers per game
`fg_pct`	REAL		Field goal percentage (0–1 or 0–100 depending on source)
`fg3_pct`	REAL		3-point percentage (0–1 or 0–100 depending on source)
`ft_pct`	REAL		Free throw percentage (0–1 or 0–100 depending on source)

*Composite PK: (`season` , `player_id` , `team_id`)

`team_game_stats`

Description: Team box score stats per game (one row per team per game per season).

Column	Type	Key	Description
`season`	TEXT	PK*	Season label
`game_id`	TEXT	PK*	Unique game identifier
`team_id`	INTEGER	PK* / FK	Team ID → `teams.team_id`
`game_date`	TEXT		Game date (stored as text)
`matchup`	TEXT		Matchup string (e.g., `CLE vs. BOS` or `CLE @ BOS`)
`wl`	TEXT		Win/Loss flag (`W` or `L`)
`pts`	INTEGER		Team points
`fgm`	INTEGER		Field goals made
`fga`	INTEGER		Field goals attempted

Column	Type	Key	Description
`fg3m`	INTEGER		3PT field goals made
`fg3a`	INTEGER		3PT field goals attempted
`ftm`	INTEGER		Free throws made
`fta`	INTEGER		Free throws attempted
`oreb`	INTEGER		Offensive rebounds
`dreb`	INTEGER		Defensive rebounds
`reb`	INTEGER		Total rebounds
`ast`	INTEGER		Assists
`stl`	INTEGER		Steals
`blk`	INTEGER		Blocks
`tov`	INTEGER		Turnovers
`plus_minus`	INTEGER		Point differential while team was on court (as provided by source)

*Composite PK: (`season` , `game_id` , `team_id`)

In [162…

```
┌──────────────────┐
│   players        │
├──────────────────┤
│ PK player_id     │
│ full_name        │
└──────────────────┘
        │
        │ FK: player_id
        ▼
┌──────────────────────────────────┐
│     player_season_stats          │
├──────────────────────────────────┤
│ PK season                        │
│ PK player_id ──────────────────► players.player_id
│ PK team_id   ──────────────────► teams.team_id
│ gp, min, pts, reb, ast, ...      │
└──────────────────────────────────┘
            │
            │ FK: team_id
            ▼
┌──────────────────────┐
│         teams        │
├──────────────────────┤
│ PK team_id           │
│ full_name            │
│ abbreviation         │
│ city, state          │
└──────────────────────┘
            │
```

```
            │ FK: team_id
            ▼
┌──────────────────────────────────┐
│        team_game_stats           │
├──────────────────────────────────┤
│ PK season                        │
│ PK game_id                       │
│ PK team_id ──────────────────────► teams.team_id
│ game_date, matchup, wl           │
│ pts, fgm, fga, fg3m, fg3a, ...   │
└──────────────────────────────────┘
```

```
  Cell In[162], line 1
    ┌──────────────┐
    ^
SyntaxError: invalid character '┌' (U+250C)
```

Setup

In [104…

```python
import pandas as pd
import sqlite3

# Connect to database
conn = sqlite3.connect('nba_5seasons.db')
print("✅ Connected to nba_5seasons.db")
```

```
✅ Connected to nba_5seasons.db
```

Let's look at the tables we have in the database first:

In SQLite, tables are stored as metadata in a system table called `sqlite_master`.

In [147…

```python
# SQL query to list all tables in the SQLite database
query = """
SELECT name
FROM sqlite_master
WHERE type='table'
ORDER BY name;
"""

# Run the query and load the results into a pandas DataFrame
tables = pd.read_sql(query, conn)

# Display the table names
tables
```

Out[147…

	name
0	player_season_stats
1	players
2	team_game_stats
3	teams

As an example, let's look at one of the tables columns/fields by using the

In [149…

```
pd.read_sql("PRAGMA table_info(player_season_stats);", conn)
```

Out[149…

	cid	name	type	notnull	dflt_value	pk
0	0	season	TEXT	0	None	1
1	1	player_id	INTEGER	0	None	2
2	2	team_id	INTEGER	0	None	3
3	3	gp	INTEGER	0	None	0
4	4	min	REAL	0	None	0
5	5	pts	REAL	0	None	0
6	6	reb	REAL	0	None	0
7	7	ast	REAL	0	None	0
8	8	stl	REAL	0	None	0
9	9	blk	REAL	0	None	0
10	10	tov	REAL	0	None	0
11	11	fg_pct	REAL	0	None	0
12	12	fg3_pct	REAL	0	None	0
13	13	ft_pct	REAL	0	None	0

In [151…

```
pd.read_sql("PRAGMA table_info(players);", conn)
```

Out[151…

	cid	name	type	notnull	dflt_value	pk
0	0	player_id	INTEGER	0	None	1
1	1	full_name	TEXT	0	None	0

In [153…

```
pd.read_sql("PRAGMA table_info(team_game_stats);", conn)
```

Out[153…

	cid	name	type	notnull	dflt_value	pk
0	0	season	TEXT	0	None	1
1	1	game_id	TEXT	0	None	2
2	2	team_id	INTEGER	0	None	3
3	3	game_date	TEXT	0	None	0
4	4	matchup	TEXT	0	None	0
5	5	wl	TEXT	0	None	0
6	6	pts	INTEGER	0	None	0
7	7	fgm	INTEGER	0	None	0
8	8	fga	INTEGER	0	None	0
9	9	fg3m	INTEGER	0	None	0
10	10	fg3a	INTEGER	0	None	0
11	11	ftm	INTEGER	0	None	0
12	12	fta	INTEGER	0	None	0
13	13	oreb	INTEGER	0	None	0
14	14	dreb	INTEGER	0	None	0
15	15	reb	INTEGER	0	None	0
16	16	ast	INTEGER	0	None	0
17	17	stl	INTEGER	0	None	0
18	18	blk	INTEGER	0	None	0
19	19	tov	INTEGER	0	None	0
20	20	plus_minus	INTEGER	0	None	0

In [155…

```
pd.read_sql("PRAGMA table_info(teams);", conn)
```

Out[155…

	cid	name	type	notnull	dflt_value	pk
0	0	team_id	INTEGER	0	None	1
1	1	full_name	TEXT	0	None	0
2	2	abbreviation	TEXT	0	None	0
3	3	nickname	TEXT	0	None	0
4	4	city	TEXT	0	None	0
5	5	state	TEXT	0	None	0
6	6	year_founded	INTEGER	0	None	0

Part 1: COUNT() - Counting Rows

COUNT() tells us how many rows match our criteria.

Example 1.1: Count All Teams

In [107…

```
# How many teams are in our database?
query = """
SELECT COUNT(*) as total_teams
FROM teams
"""

result = pd.read_sql(query, conn)
print("Total NBA Teams:")
display(result)

# COUNT(*) counts all rows in the table
```

```
Total NBA Teams:
```

	total_teams
0	30

Example 1.2: Count Player Records

In [109…

```
# How many player-season records for 2021-22?
query = """
SELECT COUNT(*) as player_records
FROM player_season_stats
WHERE season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("Player Season Records (2021-22):")
display(result)

# This counts how many players have stats recorded for that season
```

```
Player Season Records (2021-22):
```

	player_records
0	605

Key Insight: COUNT(*) counts all rows. COUNT(column) would only count non-NULL values in that column.

Part 2: SUM() - Adding Values

SUM() adds up numeric values.

Example 2.1: Lakers Total Points

In [112…

```
# Total points scored by Lakers in 2021-22
query = """
SELECT SUM(pts) as total_points
FROM team_game_stats
WHERE team_id = 1610612747
  AND season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("Lakers Total Points (2021-22 Season):")
display(result)

# SUM adds up the pts column for all Lakers games
```

```
Lakers Total Points (2021-22 Season):
```

	total_points
0	9192

Part 3: AVG() - Calculating Averages

AVG() calculates the mean value.

Example 3.1: Warriors Average Points

In [115…

```
# Warriors average points per game
query = """
SELECT ROUND(AVG(pts), 1) as avg_points_per_game
FROM team_game_stats
WHERE team_id = 1610612744
  AND season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("Warriors Average Points Per Game (2021-22):")
display(result)

# AVG calculates mean, ROUND formats to 1 decimal place
```

```
Warriors Average Points Per Game (2021-22):
```

	avg_points_per_game
0	111.0

Part 4: MIN() and MAX() - Finding Extremes

MIN and MAX find the smallest and largest values.

Example 4.1: Scoring Range

In [118…

```
# Highest and lowest scores in any game (2021-22)
query = """
SELECT
    MIN(pts) as lowest_score,
    MAX(pts) as highest_score,
    MAX(pts) - MIN(pts) as score_range
FROM team_game_stats
WHERE season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("Scoring Range for 2021-22 Season:")
display(result)

# Shows the extremes and calculates the range
```

```
Scoring Range for 2021-22 Season:
```

	lowest_score	highest_score	score_range
0	75	158	83

Part 5: Combining Multiple Aggregates

You can use multiple aggregate functions in one query.

Example 5.1: Complete Statistical Summary

In [121…

```
# Comprehensive stats for all 2021-22 games
query = """
SELECT
    COUNT(*) as total_games,
    SUM(pts) as total_points,
    ROUND(AVG(pts), 1) as avg_points,
    MIN(pts) as min_score,
    MAX(pts) as max_score
```

```
FROM team_game_stats
WHERE season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("2021-22 Season Statistics Summary:")
display(result)

# One query gives us a complete statistical overview
```

```
2021-22 Season Statistics Summary:
```

	total_games	total_points	avg_points	min_score	max_score
0	2460	272115	110.6	75	158

Key Insight: Combining aggregates gives a comprehensive summary in one query!

Part 6: GROUP BY - Category Analysis

GROUP BY splits data into groups and aggregates each separately.

Example 6.1: Games Per Team (Without GROUP BY)

In [124…

```
# Without GROUP BY - just counts ALL games
query = """
SELECT COUNT(*) as total_games
FROM team_game_stats
WHERE season = '2021-22'
"""

result = pd.read_sql(query, conn)
print("Total games across all teams:")
display(result)

# This gives us one number for the entire dataset
```

```
Total games across all teams:
```

	total_games
0	2460

Example 6.2: Games Per Team (With GROUP BY)

In [126…

```
# With GROUP BY - counts games PER TEAM
query = """
SELECT
    team_id,
    COUNT(*) as games_played
FROM team_game_stats
WHERE season = '2021-22'
```

```
GROUP BY team_id
ORDER BY games_played DESC
LIMIT 10
"""

result = pd.read_sql(query, conn)
print("Games played by each team (top 10):")
display(result)

# GROUP BY team_id creates separate groups for each team
# COUNT(*) runs on each group independently
```

```
Games played by each team (top 10):
```

	team_id	games_played
0	1610612766	82
1	1610612765	82
2	1610612764	82
3	1610612763	82
4	1610612762	82
5	1610612761	82
6	1610612760	82
7	1610612759	82
8	1610612758	82
9	1610612757	82

Example 6.3: Add Team Names with JOIN

In [128…

```
# Same query but with team names for readability
query = """
SELECT
    t.full_name as team,
    t.city,
    COUNT(tgs.game_id) as games_played
FROM teams t
JOIN team_game_stats tgs ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name, t.city
ORDER BY games_played DESC
LIMIT 10
"""

result = pd.read_sql(query, conn)
print("Games Per Team with Names (2021-22):")
display(result)

# JOIN brings in team names
# GROUP BY includes all non-aggregated columns
```

```
Games Per Team with Names (2021-22):
```

	team	city	games_played
0	Atlanta Hawks	Atlanta	82
1	Boston Celtics	Boston	82
2	Cleveland Cavaliers	Cleveland	82
3	New Orleans Pelicans	New Orleans	82
4	Chicago Bulls	Chicago	82
5	Dallas Mavericks	Dallas	82
6	Denver Nuggets	Denver	82
7	Golden State Warriors	San Francisco	82
8	Houston Rockets	Houston	82
9	Los Angeles Clippers	Los Angeles	82

Critical Rule: Every column in SELECT that's NOT an aggregate function MUST be in GROUP BY!

Part 7: HAVING - Filtering Groups

HAVING filters groups AFTER aggregation (WHERE filters rows BEFORE).

Example 7.1: WHERE vs HAVING

In [131…

```
# WHERE filters rows before grouping
query_where = """
SELECT
    team_id,
    COUNT(*) as high_scoring_games
FROM team_game_stats
WHERE season = '2021-22'
  AND pts > 120
GROUP BY team_id
ORDER BY high_scoring_games DESC
LIMIT 5
"""

result_where = pd.read_sql(query_where, conn)
print("Teams with most 120+ point games (WHERE filters games first):")
display(result_where)
```

```
Teams with most 120+ point games (WHERE filters games first):
```

	team_id	high_scoring_games
0	1610612766	30
1	1610612750	30
2	1610612749	28
3	1610612737	28
4	1610612763	27

In [132…

```
# HAVING filters groups after aggregating
query_having = """
SELECT
    team_id,
    COUNT(*) as total_games,
    ROUND(AVG(pts), 1) as avg_points
FROM team_game_stats
WHERE season = '2021-22'
GROUP BY team_id
HAVING AVG(pts) > 110
ORDER BY avg_points DESC
"""

result_having = pd.read_sql(query_having, conn)
print("Teams averaging 110+ PPG (HAVING filters groups):")
display(result_having)

# WHERE filters individual games (before grouping)
# HAVING filters teams by their average (after grouping)
```

```
Teams averaging 110+ PPG (HAVING filters groups):
```

	team_id	total_games	avg_points
0	1610612750	82	115.9
1	1610612763	82	115.6
2	1610612749	82	115.5
3	1610612766	82	115.3
4	1610612756	82	114.8
5	1610612737	82	113.9
6	1610612762	82	113.6
7	1610612759	82	113.2
8	1610612751	82	112.9
9	1610612743	82	112.7
10	1610612747	82	112.1
11	1610612738	82	111.8
12	1610612741	82	111.6
13	1610612754	82	111.5
14	1610612744	82	111.0
15	1610612758	82	110.3
16	1610612748	82	110.0

Key Difference:

- WHERE: "Show me games where [condition]"
- HAVING: "Show me teams where their aggregate [condition]"

Part 8: Exporting to Excel

Now let's export our results to Excel for further analysis.

Example 8.1: Single Sheet Export

In [135…

```
# Create a comprehensive team statistics query
query = """
SELECT
    t.full_name as team,
    t.city,
    t.state,
    COUNT(tgs.game_id) as games_played,
    ROUND(AVG(tgs.pts), 1) as avg_points,
```

```
    MIN(tgs.pts) as season_low,
    MAX(tgs.pts) as season_high
FROM teams t
JOIN team_game_stats tgs ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name, t.city, t.state
ORDER BY avg_points DESC
"""

# Execute query
team_stats = pd.read_sql(query, conn)

# Display what we're exporting
print("Team Performance Statistics:")
display(team_stats.head(10))

# Export to Excel
team_stats.to_excel('team_performance_2021_22.xlsx', index=False, sheet_name
print("\n✅ Exported to team_performance_2021_22.xlsx")
```

```
Team Performance Statistics:
```

	team	city	state	games_played	avg_points	season_low	season_
0	Minnesota Timberwolves	Minnesota	Minnesota	82	115.9	84	
1	Memphis Grizzlies	Memphis	Tennessee	82	115.6	85	
2	Milwaukee Bucks	Milwaukee	Wisconsin	82	115.5	90	
3	Charlotte Hornets	Charlotte	North Carolina	82	115.3	86	
4	Phoenix Suns	Phoenix	Arizona	82	114.8	95	
5	Atlanta Hawks	Atlanta	Georgia	82	113.9	87	
6	Utah Jazz	Utah	Utah	82	113.6	90	
7	San Antonio Spurs	San Antonio	Texas	82	113.2	90	
8	Brooklyn Nets	Brooklyn	New York	82	112.9	91	
9	Denver Nuggets	Denver	Colorado	82	112.7	85	

```
✅ Exported to team_performance_2021_22.xlsx
```

Example 8.2: Multi-Sheet Export

In [164…

```
# Create multiple related datasets
```

```
# Sheet 1: Some Basic Overall Stats
query_overall = """
SELECT
    t.full_name AS team,
    COUNT(tgs.game_id) AS games,
    SUM(tgs.pts) AS total_pts,
    SUM(tgs.fgm) AS total_fgm,
    SUM(tgs.fga) AS total_fga,
    SUM(tgs.fg3m) AS total_fg3m,
    SUM(tgs.fg3a) AS total_fg3a
FROM teams t
JOIN team_game_stats tgs ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name;
"""
overall = pd.read_sql(query_overall, conn)


# Sheet 2: Win Records
query_wins = """
SELECT
    t.full_name as team,
    SUM(CASE WHEN tgs.wl = 'W' THEN 1 ELSE 0 END) as wins,
    SUM(CASE WHEN tgs.wl = 'L' THEN 1 ELSE 0 END) as losses
FROM teams t
JOIN team_game_stats tgs ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
ORDER BY wins DESC
"""
wins = pd.read_sql(query_wins, conn)

# Sheet 3: Scoring Leaders
query_scoring = """
SELECT
    team_id,
    MAX(pts) as best_game,
    MIN(pts) as worst_game,
    MAX(pts) - MIN(pts) as consistency
FROM team_game_stats
WHERE season = '2021-22'
GROUP BY team_id
"""
scoring = pd.read_sql(query_scoring, conn)

# Export all sheets to one Excel file
with pd.ExcelWriter('nba_analysis_2021_22.xlsx', engine='openpyxl') as write
    overall.to_excel(writer, sheet_name='Overall Stats', index=False)
    wins.to_excel(writer, sheet_name='Win Records', index=False)
    scoring.to_excel(writer, sheet_name='Scoring Range', index=False)

print("\n✅ Exported to nba_analysis_2021_22.xlsx (3 sheets)")
print(f"   - Overall Stats: {len(overall)} teams")
print(f"   - Win Records: {len(wins)} teams")
print(f"   - Scoring Range: {len(scoring)} teams")
```

```
✅ Exported to nba_analysis_2021_22.xlsx (3 sheets)
   - Overall Stats: 30 teams
   - Win Records: 30 teams
   - Scoring Range: 30 teams
```

Part 9: Complete Example - Team Consistency Analysis

Let's put it all together with a comprehensive analysis.

In [171…

```
# Analyze scoring consistency across teams
# Focus: points consistency
# Shot totals are included so FG% can be calculated in Excel (not SQL)

query = """
SELECT
    t.full_name AS team,
    COUNT(tgs.game_id) AS games,
    ROUND(AVG(tgs.pts), 1) AS avg_points,
    MIN(tgs.pts) AS lowest_game,
    MAX(tgs.pts) AS highest_game,
    (MAX(tgs.pts) - MIN(tgs.pts)) AS scoring_range,
    SUM(tgs.fgm) AS total_fgm,
    SUM(tgs.fga) AS total_fga
FROM teams t
JOIN team_game_stats tgs
    ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
HAVING COUNT(tgs.game_id) >= 70
ORDER BY scoring_range ASC
LIMIT 15;
"""

# Run query and load into DataFrame
consistency = pd.read_sql(query, conn)

print("Most Consistent Teams (Smallest Scoring Range):")
display(consistency)

# Export results to Excel
consistency.to_excel(
    'team_consistency_2021_22.xlsx',
    index=False,
    sheet_name='Consistency'
)

print("\n✅ Exported to team_consistency_2021_22.xlsx")

# Notes:
# - scoring_range = MAX(points) - MIN(points)
```

```
# - smaller range = more consistent scoring
# - FG% should be calculated in Excel as: total_fgm / total_fga
```

```
Most Consistent Teams (Smallest Scoring Range):
```

	team	games	avg_points	lowest_game	highest_game	scoring_range	total_fg
0	New Orleans Pelicans	82	109.3	85	130	45	32
1	Phoenix Suns	82	114.8	95	140	45	35
2	Utah Jazz	82	113.6	90	137	47	33
3	Atlanta Hawks	82	113.9	87	136	49	34
4	Indiana Pacers	82	111.5	84	134	50	33
5	Orlando Magic	82	104.2	79	130	51	31
6	Milwaukee Bucks	82	115.5	90	142	52	34
7	Oklahoma City Thunder	82	103.7	79	134	55	31
8	Denver Nuggets	82	112.7	85	141	56	34
9	Golden State Warriors	82	111.0	82	138	56	33
10	Toronto Raptors	82	109.4	83	139	56	33
11	Detroit Pistons	82	104.8	78	134	56	31
12	Chicago Bulls	82	111.6	77	135	58	34
13	Portland Trail Blazers	82	106.2	78	136	58	31
14	Brooklyn Nets	82	112.9	91	150	59	34

✅ `Exported to team_consistency_2021_22.xlsx`

Analysis Insight: Teams with smaller scoring ranges are more predictable and consistent!

Summary

You've learned how to:

- ✅ Use aggregate functions: COUNT, SUM, AVG, MIN, MAX
- ✅ Group data with GROUP BY
- ✅ Filter groups with HAVING
- ✅ Combine aggregates for comprehensive statistics
- ✅ Export query results to Excel (single and multi-sheet)
- ✅ Understand WHERE (filter rows) vs HAVING (filter groups)

Key Patterns:

```sql
-- Pattern 1: Simple aggregation
SELECT COUNT(*), AVG(column) FROM table;

-- Pattern 2: Aggregation by category
SELECT category, COUNT(*), AVG(value)
FROM table
GROUP BY category;

-- Pattern 3: Filtered aggregation
SELECT category, COUNT(*)
FROM table
WHERE condition
GROUP BY category
HAVING COUNT(*) > threshold;
```

Next Steps:

- Complete the SQL task (write your own aggregate queries)
- Complete the Excel task (analyze the exported data)
- You're ready for Lesson 6: Database Design!

Clean Up

In [ ]:
```
# Close the database connection
conn.close()
print("✅ Connection closed")
print("\nGreat work! You understand SQL aggregations!")
```